But just like the unidirectional `@OneToMany` association, problems arise when it comes to removing the junction records from the `List`:

```
post1.getTags().remove(tag1);
```

Hibernate deletes all junction rows associated with the `Post` entity whose `Tag` association is being removed and inserts back the remaining ones:

```
DELETE FROM post_tag WHERE post_id = 1
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
```

10.6.2 Unidirectional @ManyToMany Set

When using a `List`, the `@ManyToMany` association behaves like a bag, and, as explained in the `@ElementCollection` section, Hibernate bags are not very efficient.

Therefore, let's change the `tags` collection to use a `Set`:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();
```

Assuming, we added the same 3 `Tag` entities to the `Post` entity, if we remove the first `Tag` from the `tags` collection:

```
post1.getTags().remove(tag1);
```

Hibernate is going to generate a single SQL delete statement, this time:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 2
```

For a `@ManyToMany` association, it's better to use the `Set` collection type, instead of `List`.

10.6.3 Bidirectional @ManyToMany

The bidirectional `@ManyToMany` relationship can be navigated from both the `Post` and the `Tag` side.

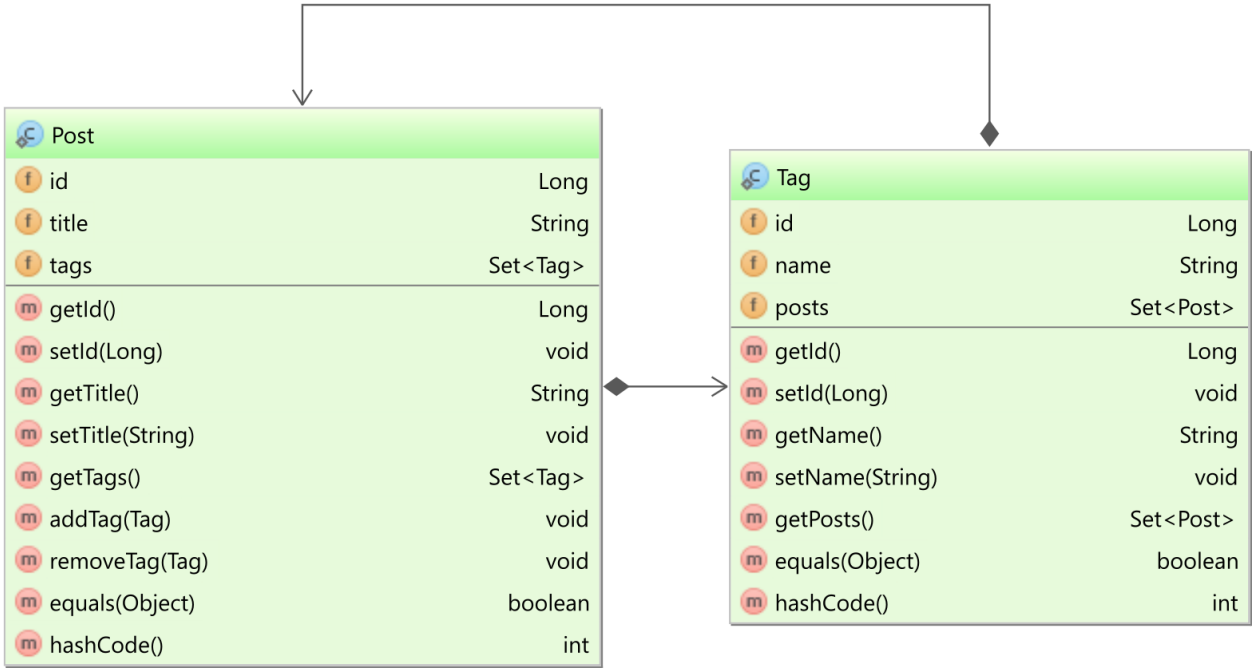

Figure 10.15: The bidirectional @ManyToMany relationship

While in the one-to-many and many-to-one associations the child-side is the one holding the foreign key, for a many-to-many table relationship both ends are parent-sides and the junction table plays the child-side role.

Because the junction table is hidden when using the default `@ManyToMany` mapping, the application developer must choose an owning and a `mappedBy` side.

The `Post` entity uses the same mapping as shown in the unidirectional `@ManyToMany` section:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();
```

while the `Tag` entity adds a `@ManyToMany` association that looks as follows:

```
@ManyToMany(mappedBy = "tags")
private Set<Post> posts = new HashSet<>();
```

The `mappedBy` attribute tells Hibernate that the `Post` entity is the owning side of this association. In our case, the `mappedBy` attribute of the `@ManyToMany` annotation in the `Tag` entity points to the `tags` collection of the `Post` entity, meaning that the `Post` entity is going to propagate the association state changes to the underlying junction table.

Hibernate manages each side of a bidirectional `@ManyToMany` relationship just like a unidirectional `@OneToMany` association between the parent-side (e.g. `Post` or the `Tag`) and the hidden child-side (e.g. the `post_tag` table `post_id` or `tag_id` foreign keys).

For this reason, it's much more efficient to use a `Set` collection type, no matter if the `@ManyToMany` is unidirectional or bidirectional.

Like any bidirectional associations, helper methods are needed to keep both parent sides in sync. For a bidirectional `@ManyToMany` association, the helper methods must be added to the entity that is more likely to interact with. In our case, the root entity is the `Post`, so the helper methods are added to the `Post` entity:

```
public void addTag(Tag tag) {
    tags.add(tag);
    tag.getPosts().add(this);
}

public void removeTag(Tag tag) {
    tags.remove(tag);
    tag.getPosts().remove(this);
}
```

Both `Post` and `Tag` entities have unique attributes which can simplify the entity removal operation even when mixing detached and managed entities.

10.6.4 The @OneToMany alternative

We can also map a many-to-many table relationship using bidirectional `@OneToMany` associations from the parent entities to the child entity that maps the junction table.

Therefore, the `post_tag` child table is mapped by the `PostTag` entity, and its composite identifier is encapsulated in the `PostTagId` embeddable type as illustrated by the following diagram.

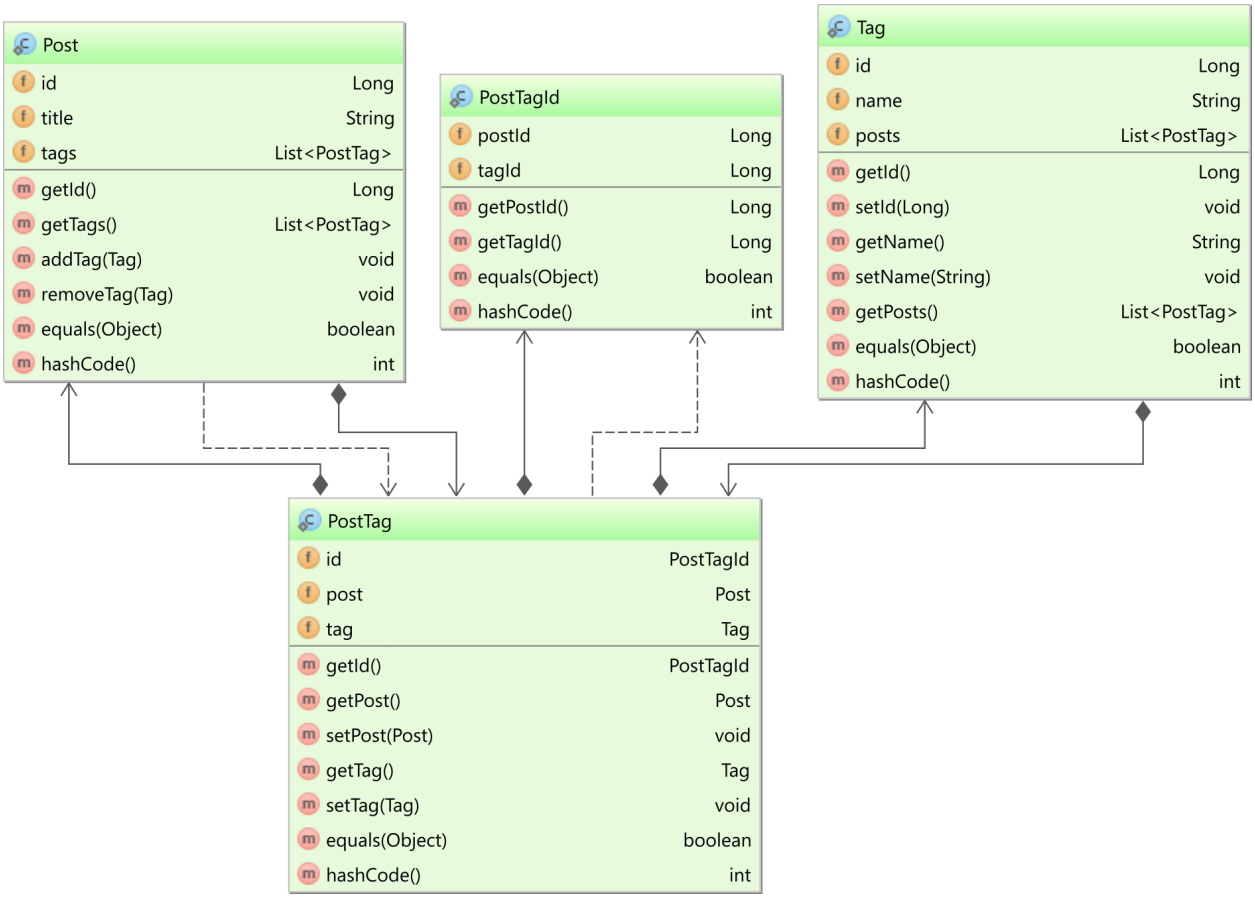

Figure 10.16: The @OneToMany as a many-to-many table relationship

The `PostTagId` maps the `postId` and `tagId` foreign key columns, and the `PostTag` entity uses the `postId` and `tagId` properties of the `PostTagId` identifier to map the `post` and the `tag` `@ManyToOne` associations.

The `Post` entity maps the bidirectional `@OneToMany` side of the `post` `@ManyToOne` association from the `PostTag` child entity:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> tags = new ArrayList<>();
```

The `Tag` entity maps the bidirectional `@OneToMany` side of the `tag` `@ManyToOne` association defined by the `PostTag` child entity:

```
@OneToMany(mappedBy = "tag", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> posts = new ArrayList<>();
```

This way, the bidirectional `@ManyToMany` relationship is transformed in two bidirectional `@OneToMany` associations.

The `PostTag` entity has a composed identifier made out of the `post_id` and `tag_id` columns.

```
@Embeddable
public class PostTagId implements Serializable {

    @Column(name = "post_id")
    private Long postId;

    @Column(name = "tag_id")
    private Long tagId;

    public PostTagId() {}

    public PostTagId(Long postId, Long tagId) {
        this.postId = postId;
        this.tagId = tagId;
    }

    public Long getPostId() {
        return postId;
    }

    public Long getTagId() {
        return tagId;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTagId that = (PostTagId) o;
        return Objects.equals(postId, that.getPostId() &&
                Objects.equals(tagId, that.getTagId());
    }

    @Override
    public int hashCode() {
        return Objects.hash(postId, tagId);
    }
}
```

The JPA specification requires the composite identifiers embeddable types to implement the `Serializable` interface, as well as implementing `equals` and `hashCode` based on the associated foreign key column values.